MATH317 · NUMERICAL METHODS

Chapter 2 Floating-Point

Midterm summary · Lecture 4

Muharib

What's inside

01 Why computers cannot store every number

02 Normalized form

03 Single and Double precision

04 Converting to single precision: seven steps

05 The same number in double precision

06 Going backwards: hex to decimal

07 Roundoff error

08 How it shows up in the midterm

09 Common mistakes

10 Check yourself

01 Why computers cannot store every number

You write numbers in base 10. A computer stores them in base 2 and has only a fixed number of bits.

The problem: some numbers are short in decimal but never end in binary.

$$(0.1)_{10} = (0.0\ 0011\ 0011\ 0011\ \ldots)_2$$

Machine numbers

The numbers a computer can store exactly are called **Machine numbers**. There are finitely many, and most real numbers are not among them.

Term	Meaning
Overflow	The number is larger than the largest value the machine can store.
Underflow	The number is closer to zero than the smallest value the machine can store.
Hole at zero	The gap between zero and the smallest positive stored number.

02 Normalized form

Before the bits, write the number in one fixed shape: move the point and record how far it

moved.

```
Decimal:   x = ± 0.d₁d₂d₃... × 10ⁿ        (d₁ ≠ 0)
```

```
Example:   79325 = 0.79325 × 10⁵
```

```
Computer:  x = ± 1.xxxx × 2ᵐ
```

The part `0.79325` is the **Mantissa** and the power `5` is the **Exponent**.

Watch out

In the exam always move the point until exactly one `1` is in front of it. The slides call this the **one-plus form**.

The toy machine

The slides imagine a machine whose numbers all have this form:

```
x = ± (0.b₁b₂b₃)₂ × 2^(±k)        every bᵢ and k is 0 or 1
```

If we require `b₁ = 1`, the smallest positive number is $(0.100)_2 \times 2^{-1} = \frac{1}{4}$. Nothing lies between `0` and $\frac{1}{4}$; that gap is the **Hole at zero**.

03 Single and Double precision

Single precision (32-bit)




```
value = (−1)ˢ × 2^(c − 127) × (1.f)₂
```

Double precision (64-bit)



$$value = (-1)^s \times 2^{(c - 1023)} \times (1.f)_2$$

Field	Name	What it holds
`s`	**Sign**	The sign: `0` positive, `1` negative.
`c`	**Biased exponent**	The exponent after adding the **Bias**.
`f`	**Fraction** / **Mantissa**	Only the bits after the point. The leading 1 is not stored.

Comparison table (memorize it)

	Single	Double
Exponent bits	8	11
Fraction bits	23	52
Bias	127	1023
Largest number	$\approx 3.4 \times 10^{38}$	$\approx 1.8 \times 10^{308}$
Smallest positive	$\approx 1.2 \times 10^{-38}$	$\approx 2.2 \times 10^{-308}$
Machine epsilon	$2^{-23} \approx 1.2 \times 10^{-7}$	$2^{-52} \approx 2.2 \times 10^{-16}$
Hex digits in the answer	8	16

Machine epsilon

The smallest positive number ε for which the computer still sees `1 + ε` as different from `1`. It measures the machine's precision.

04 Converting to single precision: seven steps

This is the exam question of the chapter. It appeared in four midterms. We solve the slide example `−52.234375` step by step.

1 SIGN The sign

The number is negative, so `s = 1`. From here on work with `52.234375`.

2 INTEGER PART Integer part to binary

The doctor converts to octal first, because each octal digit is exactly 3 bits. Divide by 8 and read the remainders from bottom to top.

```
52 ÷ 8 = 6    remainder 4
 6 ÷ 8 = 0    remainder 6
```

$52 = (64)_8$ 6 → 110 4 → 100

$52 = (110100)_2$

3 FRACTION PART Fraction to binary

For the fraction multiply by 8 and take the integer part each time, reading from top to bottom.

```
0.234375 × 8 = 1.875   →  1
```

```
0.875    × 8 = 7.000   →  7
```

```
0.234375 = (0.17)₈      1 → 001      7 → 111
```

```
0.234375 = (0.001111)₂
```

4 NORMALIZE Move the point

```
52.234375 = (110100.001111)₂ = 1.10100001111 × 2⁵
```

The point moved 5 places to the left, so `m = 5`.

5 EXPONENT Add the bias

```
c = 5 + 127 = 132 = (204)₈ = (10000100)₂
```

It must be exactly 8 bits.

6 FRACTION FIELD Fill the f field

Take only the bits after the point and pad with zeros on the right to 23 bits.

```
f = 10100001111 00000000000
```

7 HEX **Assemble and convert to hex**

```
1 | 10000100 | 10100001111000000000000
```

```
1100 0010 0101 0000 1111 0000 0000 0000
```

```
 C    2    5    0    F    0    0    0
```

$-52.234375 = [C250F000]_{16}$

Check your answer

A negative number always starts with a hex digit from 8 to F. A single-precision answer always has 8 hex digits.

Hex table

Bits	Hex	Bits	Hex	Bits	Hex	Bits	Hex
0000	0	0100	4	1000	8	1100	C
0001	1	0101	5	1001	9	1101	D
0010	2	0110	6	1010	A	1110	E
0011	3	0111	7	1011	B	1111	F

05 The same number in double precision

Steps 1 to 4 are identical. What changes: the bias, the exponent width and the fraction

width.

```
c = 5 + 1023 = 1028 = (2004)₈ = (10000000100)₂          (11 bits)
```

```
1 | 10000000100 | 10100001111 000...0
```

```
1100 0000 0100 1010 0001 1110 0000 ... 0000
```

```
−52.234375 = [C04A1E0000000000]₁₆
```

Watch out

Do not reuse the hex digits from the single-precision answer. The exponent is now 11 bits, so every group of 4 shifts.

06 Going backwards: hex to decimal

The same steps in reverse. Slide example `[45DE4000]₁₆`:

```
 4    5    D    E    4    0    0    0
```

```
0100 0101 1101 1110 0100 0000 0000 0000
```

```
s = 0        c = 10001011 = 139        f = 101111001 000...
```

```
m = 139 − 127 = 12
```

```
value = (1.101111001)₂ × 2¹² = (1101111001000)₂
```

```
      = 4096 + 2048 + 512 + 256 + 128 + 64 + 8
```

```
      = 7112
```

07 Roundoff error

When a number does not fit in the `f` field, the computer stores the nearest **Machine number**. The difference is the **Roundoff error**.

$$fl(x) = x\,(1+\delta) \qquad |\delta| \le u$$

`fl(x)` is the number actually stored. `u` is the **Unit roundoff**: the largest possible relative error from one rounding.

	Single	Double
Unit roundoff (u)	2^{-24}	2^{-53}
Machine epsilon (ε)	2^{-23}	2^{-52}

Slide example

A machine keeps 5 decimal digits. Add $x = 0.37218 \times 10^{4}$ and $y = 0.71422 \times 10^{-1}$.

$$x = 0.3721800000 \times 10^{4}$$

$$y = 0.0000071422 \times 10^{4}$$

$$x + y = 0.3721871422 \times 10^{4}$$

$$fl(x+y) = 0.37219 \times 10^{4}$$

$$\text{relative error} = \frac{|3721.871422 - 3721.9|}{3721.871422}$$

$$\approx 7.7 \times 10^{-6}$$

Loss of significance

Subtracting two nearly equal numbers destroys most of the correct digits and inflates the relative error. This is **Loss of significance**.

08 How it shows up in the midterm

Midterm	Number	Format	Answer
472	−23.21875	Single	C1B9C000
471	−45.21875	Single	C234E000
461	1259.125	Single	449D6400
462	−184.125	Double	C067040000000000

The question is worth 6 to 7 marks, split across the steps. The answer key states that skipping any step loses marks.

Past papers show the style of the question. They are not a promise about the next exam.

09 Common mistakes

Mistake	Fix
Storing the leading 1 inside `f`	`f` holds only the bits after the point.
Using `127` in a double-precision question	Double uses `1023` and an 11-bit exponent.
Forgetting the sign bit	Do it first. Negative means `s = 1`.
Writing the exponent with fewer than 8 bits	Pad with zeros on the left. Example: 124 is `01111100`.
A number below 1 with a positive exponent	If the point moves right, the exponent is negative.

10 Check yourself

1. Represent `85.125` in single precision.
2. Represent `-0.15625` in single precision.
3. Represent `13.5` in double precision.
4. Which decimal number is stored as `[C1500000]16`?

Solutions

1) $85 = (125)_8 = (1010101)_2$ $\quad .125 = (.1)_8 = (.001)_2$

$1010101.001 = 1.010101001 \times 2^6$

$c = 6 + 127 = 133 = (10000101)_2$

0 | 10000101 | 01010100100000000000000

$= [42AA4000]_{16}$

2) $0.15625 = (.00101)_2 = 1.01 \times 2^{-3}$

$c = -3 + 127 = 124 = (01111100)_2$

1 | 01111100 | 01000000000000000000000

$= [BE200000]_{16}$

3) $13.5 = (1101.1)_2 = 1.1011 \times 2^3$

$c = 3 + 1023 = 1026 = (10000000010)_2$

0 | 10000000010 | 1011 000...0

$= [402B000000000000]_{16}$

4) 1100 0001 0101 0000 ... 0000

$s = 1$ $\quad c = 10000010 = 130$ $\quad m = 3$ $\quad f = 101$

$value = -(1.101)_2 \times 2^3 = -(1101)_2$

$= -13$